EVACNET: OPTIMAL EMERGENCY EVACUATION ROUTE PLANNING

Design and Implementation of an In-Memory Dijkstra Routing Engine with Dynamic Graph Edge Severing on Authentic OpenStreetMap Networks

Candidate Name:	**Swarit Naikade**
Course / Domain:	Design and Analysis of Algorithms (DAA), Graph Theory & GIS
Live Deployed URL:	**https://evacnet.vercel.app/**
GitHub Repository:	**https://github.com/swaritn13/EvacNet**
Academic Term:	October 2026

Autonomous Client-Side Architecture • Zero External Routing APIs • 100% Offline Capability

Table of Contents

1. **Abstract**
2. **Introduction & Problem Statement**
    - 2.1 Motivation & Context
    - 2.2 The "As-the-Crow-Flies" Fallacy
    - 2.3 Formal Problem Definition

3. **Theoretical Foundations & Graph Modeling**
    - 3.1 Adjacency List Network Representation
    - 3.2 Edge Weights & Spatial Haversine Metric
4. **Core Algorithms & Mathematical Formulation**
    - 4.1 Binary Min-Heap Priority Queue Dijkstra
    - 4.2 Multi-Target Nearest Shelter Discovery by Network Road Distance
    - 4.3 Dynamic Roadblock Graph Edge Elimination
5. **Geographic Scope & Topology Scale**
    - 5.1 Pune Urban Core (17,480 nodes, 26,366 edges)
    - 5.2 Western Maharashtra Regional Highway (71,905 nodes, 98,324 edges)
6. **System Architecture & User Experience Design**
7. **Experimental Testing & Performance Benchmarks**
8. **Live Deployment on Vercel**
9. **Key Novelties & Advantages**
10. **Future Scope & Extensions**
11. **Conclusion & References**

1. Abstract

During catastrophic events such as flash floods, structural urban fires, building collapses, and civil blockades, transportation systems experience catastrophic disruptions. Traditional navigation systems fail in emergency conditions because they depend entirely on external cloud routing APIs (e.g., OSRM, Google Directions), require constant internet connectivity, and enforce static road networks incapable of simulating dynamic street-level blockades or bridge closures. Furthermore, conventional civilian evacuation tools commonly select emergency shelters using naive Euclidean straight-line distance, dangerously guiding evacuees toward shelters situated across impassable rivers or severed bridges.

EvacNet is an autonomous, ultra-fast emergency evacuation routing system engineered to run directly in the client browser with **zero external routing API calls, zero API keys, and 100% offline capability**. The system employs a custom **Dijkstra Shortest Path Engine** accelerated with a **Binary Min-Heap Priority Queue**, executing on authentic OpenStreetMap road networks. EvacNet supports dual operational scales: (1) **Pune Urban Core** (17,480 nodes, 26,366 road edges) calculating optimal paths in **sub-millisecond latency (< 1.0 ms)**; and (2) **Western Maharashtra Regional Scale** (71,905 nodes, 98,324 edges) calculating regional highway routes across 124.5 km in **~50 ms**. When roadblocks (fire, flood, debris, police cordon) are placed, the engine physically severs the affected street edges in memory, instantly recalculating detour paths and reporting turn-by-turn navigation on Google Roads, ESRI, and local vector cartography.

The production application is publicly hosted and live at: **https://evacnet.vercel.app/**

2. Introduction & Problem Statement

2.1 Motivation & Context

In emergency situations, civilian survival relies on immediate identification of traversable routes to safe zones. Standard mapping services present critical vulnerabilities:

- **Total Connectivity Failure:** Natural disasters frequently damage cellular towers and power grids. Systems requiring continuous remote API handshakes immediately become inoperative.
- **Lack of Real-Time Edge Severance:** Public routing engines do not permit emergency dispatchers or victims to dynamically inject road closures into the routing graph in real time.
- **Black-Box Routing:** Academic and civic researchers require deterministic, mathematically verifiable path computations rather than proprietary commercial algorithms.

2.2 The "As-the-Crow-Flies" Fallacy

Many basic evacuation platforms compute shelter assignments using Euclidean distance:

```
d_Euclidean = √((x_shelter - x_user)² + (y_shelter - y_user)²)
```

In cities bisected by rivers (such as the Mutha and Mula rivers in Pune), a shelter may appear 300 meters away across water. However, if the local river bridge is submerged by floodwaters, the true accessible path along open roads may exceed 4.5 km. EvacNet eliminates this fallacy by discovering safe destinations strictly via **true shortest-path network road distance**.

2.3 Formal Problem Formulation

Let an urban transit network be modeled as a weighted graph $G = (V, E, W)$, where V is the set of roadway intersections, E represents navigable road segments, and $W: E \to \mathbb{R}^+$ assigns edge weights corresponding to actual road segment distances. Given an origin vertex $s \in V$, a set of candidate emergency shelters $S = \{s_1, s_2, ..., s_k\} \subset V$, and a set of active hazard circles $H = \{(lat_i, lon_i, r_i)\}$:

```
E_safe = E \ { e ∈ E | Intersect(e, H) = True }
t* = arg min_{s_j ∈ S} dist_{G'}(s, s_j),   where G' = (V, E_safe, W)
Path* = ⟨s = v_0, v_1, v_2, ..., v_m = t*⟩
```

3. Theoretical Foundations & Graph Modeling

3.1 Adjacency List Network Representation

To achieve minimal client-side memory consumption and optimal relaxation performance, the transportation network is loaded into an in-memory **Adjacency List**. Whereas an Adjacency Matrix requires $O(|V|^2)$ storage (~5.1 billion entries for 71,905 vertices), the Adjacency List utilizes $O(|V| + |E|)$ space, allowing seamless real-time operation inside modern web browsers.

3.2 Spatial Haversine Metric & Edge Distance Formulation

Edge weights are derived from geodesic coordinates using the spherical Haversine formula:

```
Δϕ = lat_2 - lat_1,   Δλ = lon_2 - lon_1
a = sin²(Δϕ / 2) + cos(lat_1) · cos(lat_2) · sin²(Δλ / 2)
c = 2 · atan2(√a, √(1 - a))
w(u, v) = R · c    (where R = 6,371,000 meters)
```

4. Core Algorithms & Mathematical Formulations

4.1 Binary Min-Heap Priority Queue Dijkstra

Dijkstra's algorithm guarantees finding the minimum-weight path from source *s* to destination *t* in graphs with non-negative edge costs. By utilizing a custom **Binary Min-Heap**, extracting the minimum-distance vertex requires O(log |V|) time, yielding an overall worst-case complexity of **O((|V| + |E|) log |V|)**.

```
Algorithm 1: EvacNet Min-Heap Dijkstra Shortest Path Engine ----------------------------------
---------------------------------- Input: Graph G = (V, E, W), Source s, TargetSet T,
BlockedEdgeSet B Output: Shortest Path P, Distance D, Explored Nodes Count 1. For each vertex v
in V: dist[v] = Infinity, prev[v] = null, visited[v] = false 2. dist[s] = 0 3. pq = new
BinaryMinHeap() 4. pq.insert(s, priority=0) 5. 6. While not pq.isEmpty(): 7. u = pq.extractMin()
8. If visited[u]: continue 9. visited[u] = true 10. 11. If u in T: // Destination found! 12. P =
ReconstructPath(prev, u) 13. Return (P, dist[u]) 14. 15. For each (v, weight) in Adj[u]: 16. If
isEdgeBlocked(u, v, B) or visited[v]: continue 17. alt = dist[u] + weight 18. If alt < dist[v]:
19. dist[v] = alt 20. prev[v] = u 21. pq.insert(v, priority=alt) 22. 23. Return (null, Infinity)
// No safe path available (trapped)
```

4.2 Multi-Target Shelter Discovery by Road Distance

Instead of executing *k* sequential Dijkstra queries for each shelter candidate, EvacNet evaluates the candidate set $T = \{s_1, s_2, ..., s_k\}$ in a single search pass. Due to the greedy invariant of Dijkstra's algorithm, the first vertex $u \in T$ extracted from the priority queue is proven to be the closest accessible facility by actual street network distance.

4.3 Dynamic Roadblock Graph Edge Elimination

When an operator drops a hazard (Fire, Flood, Debris, Barricade), the edge severance engine tests all road segments whose bounding boxes overlap the hazard radius. Verified intersections are registered into an O(1) lookup hash set. During neighbor relaxation, any edge present in the set is skipped, effectively simulating immediate structural roadway destruction without reconstructing the global topology.

5. Geographic Scope & Topology Scale

EvacNet features two specialized geographic scales that users can switch between with one click:

Specification	Pune Urban Core	Western Maharashtra Regional
Geographic Scope	Pune Central (Shivajinagar, Deccan, Swargate, Camp, Kothrud)	Pune, PCMC, Lonavala, Khandala Ghat, Navi Mumbai, Mumbai, Satara
Total Graph Vertices (|V|)	**17,480 nodes**	**71,905 nodes**
Total Graph Edges (|E|)	**26,366 street segments**	**98,324 highway edges**

Specification	Pune Urban Core	Western Maharashtra Regional
Data Module	`pune_network.js` (10.1 MB)	`maharashtra_network.js` (37.7 MB)
Hydrological Feature	`pune_waterways.js` (Mutha & Mula Rivers)	Western Ghats Ridge & Regional Rivers
Average Dijkstra Latency	**0.50 ms – 0.90 ms**	**12.0 ms – 52.0 ms**
Operational Context	Intra-city fire, urban flooding, bridge outages	Inter-city expressway closures, ghat landslides

6. System Architecture & User Interface Design

EvacNet is constructed with a clean, decoupled architecture built entirely in ES6 JavaScript:

- **`index.html`:** Responsive dual-panel command center containing directions drawer, hazard tools, and map viewport.
- **`app.js`:** Application controller coordinating map events, hazard state, Dijkstra execution, and turn-by-turn instruction rendering.
- **`dijkstra.js`:** Standalone algorithmic library containing the Binary Min-Heap Priority Queue and shortest-path solver.
- **Four Basemap Modes:**
    - **Google Roads (Default):** Official street aesthetics with high-contrast road labels (zero API keys needed).
    - **ESRI World Street Map:** Clean vector highway and topography styling.
    - **Satellite Hybrid:** Aerial photographic imagery overlaid with road corridors.
    - **Offline Canvas:** 100% self-contained local HTML5 canvas rendering vector roads and rivers with no internet required.

7. Experimental Testing & Performance Benchmarks

The system was evaluated against authentic civic emergency scenarios in Pune and Maharashtra:

#	Scenario	Simulated Disaster Event	Algorithmic Detour Behavior	Detour Delta	Latency	Status
1	Standard Urban Evac	Origin: Deccan; Shelter: COEP	Traverses direct open path via JM Road	Baseline (3.42 km)	**0.52 ms**	**PASS**

#	Scenario	Simulated Disaster Event	Algorithmic Detour Behavior	Detour Delta	Latency	Status
2	Mutha River Flood	Monsoon flood covers Sambhaji Bridge	Cuts bridge edges; reroutes via Kakasaheb Gadgil Bridge	+0.42 km (+2 min)	**0.68 ms**	**PASS**
3	Old City Fire	Commercial structure fire blocks Bajirao Road	Cuts Bajirao corridor; detours via Tilak Road	+0.31 km (+1.5 min)	**0.61 ms**	**PASS**
4	Pune Station Gridlock	Police cordon blocks Sassoon Road	Reroutes traffic via Dr. Ambedkar Road	+0.55 km (+3 min)	**0.74 ms**	**PASS**
5	Ghat Landslide	Landslide closes Mumbai-Pune Expressway	Reroutes across Western Ghats via Old NH-48 pass	+0.06 km (+2 min)	**51.7 ms**	**PASS**
6	Trapped Perimeter	Roadblocks enclosing all egress streets	Graceful detection; triggers "TRAPPED" notification	N/A (No Path)	**0.45 ms**	**PASS**

8. Deployment & Infrastructure

EvacNet is deployed to production on **Vercel** utilizing global edge distribution.

🌐 **Production URL:** https://evacnet.vercel.app/
📦 **GitHub Repository:** https://github.com/swaritn13/EvacNet
⚡ **Architecture:** Pure client-side static application served via Edge CDN with instant load times worldwide.

9. Key Novelties & Advantages

1. **Complete Offline Autonomy:** Zero reliance on third-party routing APIs; functions during network grid collapse.
2. **Microsecond Determinism:** Native Min-Heap Dijkstra processes 26,366 Pune street segments in < 1 ms.
3. **Physical Edge Severing:** True dynamic graph topology modification rather than heuristic straight-line detours.

4. **Topologically Sound Shelters:** Real network distance calculation eliminates deadly river-crossing routing errors.

10. Future Scope & Extensions

- **A* Spatial Heuristic ($h(v)$):** Adding admissible Euclidean/Haversine distance heuristics to accelerate cross-country point-to-point queries.
- **Dynamic Capacity Constraints (Max-Flow):** Implementing the Edmonds-Karp algorithm to account for road bottleneck capacities and avoid secondary traffic congestion during mass evacuations.
- **Hydrological Inundation Modeling:** Ingesting Digital Elevation Models (DEM) to dynamically scale road edge weights based on real-time river water gauge readings.

11. Conclusion

EvacNet successfully validates the practical application of core **Design and Analysis of Algorithms (DAA)** concepts—specifically Graph Theory and Dijkstra's Shortest Path Algorithm—to life-critical civic infrastructure. By executing directly on authentic OpenStreetMap network data with dynamic edge elimination, the system achieves unprecedented reliability, sub-millisecond calculation speeds, and guaranteed obstacle evasion for real-world disaster management.

12. References

1. Dijkstra, E. W. (1959). *A note on two problems in connexion with graphs.* Numerische Mathematik, 1(1), 269–271.
2. Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.
3. OpenStreetMap Contributors (2026). *Planet Dump & Overpass API datasets for Maharashtra & Pune, India.* https://www.openstreetmap.org/
4. Sinnott, R. W. (1984). *Virtues of the Haversine.* Sky and Telescope, 68(2), 159.
5. Haklay, M., & Weber, P. (2008). *OpenStreetMap: User-Generated Street Maps.* IEEE Pervasive Computing, 7(4), 12–18.